

```

import { useState, useEffect, useMemo } from "react";

const COLORS = [
  "#ef4444", "#f97316", "#f59e0b", "#eab308", "#84cc16", "#22c55e",
  "#10b981", "#14b8a6", "#06b6d4", "#3b82f6", "#60a5fa", "#6366f1",
  "#8b5cf6", "#a855f7", "#d946ef", "#ec4899", "#f43f5e"
];

const today = new Date();

function getDaysAgo(n) {
  const d = new Date(today);
  d.setDate(d.getDate() - n);
  return d;
}

function dateKey(d) {
  return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2, '0')}-${String(d
}

const todayKey = dateKey(today);
const yesterdayKey = dateKey(getDaysAgo(1));

const DEFAULT_HABITS = [
  { id: 1, name: "Sleeping", color: "#ef4444", completions: { [yesterdayKey]: tru
  { id: 2, name: "Prayer", color: "#eab308", completions: { [yesterdayKey]: true,
  { id: 3, name: "Meditation 10 min", color: "#84cc16", completions: {} } },
];

function loadHabits() {
  try { const r = localStorage.getItem("ritual-habits"); if (r) return JSON.parse
  return null;
}

function saveHabits(h) {
  try { localStorage.setItem("ritual-habits", JSON.stringify(h)); } catch (_) {}
}

function loadDarkMode() {
  try { const r = localStorage.getItem("ritual-darkmode"); if (r !== null) return
  return true;
}

function loadReminder() {
  try { const r = localStorage.getItem("ritual-reminder"); if (r !== null) return
  return { enabled: false, time: "08:00" };
}

```

```
// Theme
```

```
function makeTheme(dark) {  
  return dark ? {  
    bg: "#111711", sidebar: "#141a13", card: "#1a2219", cardAlt: "#242e23",  
    border: "#1e2a1d", border2: "#2e3e2c", text: "#e8e0d4", muted: "#6b7a6a",  
    subtle: "#a0a89e", faint: "#4a5a49", faint2: "#2a3028", accent: "#c47a5a",  
    accentBtn: "#8b6a4a", navActive: "#2a3828", navActiveTxt: "#c8d4b8",  
    navTxt: "#7a8a78", toggleBg: "#2a3828", toggleDot: "#4a5a49",  
    modalBg: "#1e2a1d", inputBg: "#242e23",  
  } : {  
    bg: "#f5f5f0", sidebar: "#ebebe4", card: "ffffff", cardAlt: "#f0f0ea",  
    border: "#deded8", border2: "#d0d0ca", text: "#1a2219", muted: "#6b7060",  
    subtle: "#5a5a50", faint: "#9a9a90", faint2: "#d0d0ca", accent: "#c47a5a",  
    accentBtn: "#8b6a4a", navActive: "#deded8", navActiveTxt: "#1a2219",  
    navTxt: "#6b7060", toggleBg: "#deded8", toggleDot: "#9a9a90",  
    modalBg: "ffffff", inputBg: "#f5f5f0",  
  };  
}
```

```
// Helpers
```

```
function CircularProgress({ pct, color = "#ef4444", size = 80, t }) {  
  const r = 32, cx = 40, cy = 40, circ = 2 * Math.PI * r;  
  const dash = (pct / 100) * circ;  
  return (  
    <svg width={size} height={size} viewBox="0 0 80 80">  
      <circle cx={cx} cy={cy} r={r} fill="none" stroke={t.faint2} strokeWidth="6"  
      <circle cx={cx} cy={cy} r={r} fill="none" stroke={color} strokeWidth="6"  
        strokeDasharray={` ${dash} ${circ - dash}` } strokeDashoffset={circ / 4}  
        strokeLinecap="round" style={{ transition: "stroke-dasharray 0.5s ease" }}  
      <text x="50%" y="50%" dominantBaseline="middle" textAnchor="middle"  
        fill={t.text} fontSize="13" fontWeight="700">{pct}%</text>  
    </svg>  
  );  
}
```

```
function Toggle({ on, onToggle, t }) {  
  return (  
    <div onClick={onToggle} style={{  
      width: 44, height: 24, background: on ? t.accent : t.toggleBg,  
      borderRadius: 12, cursor: "pointer", position: "relative", transition: "bac  
    }}>  
      <div style={{  
        width: 18, height: 18, background: on ? "fff" : t.toggleDot,  
        borderRadius: "50%", position: "absolute", top: 3,
```

```

        left: on ? 23 : 3, transition: "left 0.25s"
      }} />
    </div>
  );
}

function streak(habit) {
  let s = 0, d = new Date(today);
  while (habit.completions[dateKey(d)]) { s++; d.setDate(d.getDate()-1); }
  return s;
}

function longestStreak(habit) {
  const keys = Object.keys(habit.completions).sort();
  if (!keys.length) return 0;
  let max = 0, cur = 0, prev = null;
  for (const k of keys) {
    if (!habit.completions[k]) continue;
    if (prev) { const diff = (new Date(k) - new Date(prev)) / 86400000; cur = dif
    else cur = 1;
    if (cur > max) max = cur;
    prev = k;
  }
  return max;
}

function rateForDays(habit, days) {
  let count = 0;
  for (let i = 0; i < days; i++) if (habit.completions[dateKey(getDaysAgo(i))]) c
  return Math.round((count / days) * 100);
}

function last7Days() { return Array.from({ length: 7 }, (_, i) => getDaysAgo(6 -

// Notifications

async function requestNotificationPermission() {
  if (!("Notification" in window)) return "unsupported";
  if (Notification.permission === "granted") return "granted";
  if (Notification.permission === "denied") return "denied";
  const result = await Notification.requestPermission();
  return result;
}

function scheduleReminder(time, habits) {
  // Clear any existing scheduled reminders
  const existingId = parseInt(localStorage.getItem("ritual-reminder-timeout")) ||
  if (existingId) clearTimeout(existingId);

  const [hours, minutes] = time.split(":").map(Number);

```

```

const now = new Date();
const next = new Date();
next.setHours(hours, minutes, 0, 0);
if (next <= now) next.setDate(next.getDate() + 1);

const msUntil = next - now;

const id = setTimeout(() => {
  const incomplete = habits.filter(h => !h.completions[todayKey]);
  if (incomplete.length > 0 && Notification.permission === "granted") {
    new Notification("Ritual - Daily Reminder", {
      body: `You have ${incomplete.length} habit${incomplete.length > 1 ? "s" : ""}`,
      icon: "https://img.icons8.com/fluency/96/leaf.png",
      badge: "https://img.icons8.com/fluency/96/leaf.png",
    });
  }
  // Reschedule for next day
  scheduleReminder(time, habits);
}, msUntil);

localStorage.setItem("ritual-reminder-timeout", String(id));
}

function cancelReminder() {
  const id = parseInt(localStorage.getItem("ritual-reminder-timeout") || "0");
  if (id) { clearTimeout(id); localStorage.removeItem("ritual-reminder-timeout"); }
}

// Sidebar

function Sidebar({ active, setActive, dark, setDark, t }) {
  const nav = [
    { id: "dashboard", label: "Dashboard", icon: "[]" },
    { id: "habits", label: "Habits", icon: "=" },
    { id: "statistics", label: "Statistics", icon: "^" },
    { id: "calendar", label: "Calendar", icon: "#" },
  ];

  return (
    <div style={{
      width: 200, background: t.sidebar, padding: "20px 12px",
      display: "flex", flexDirection: "column", gap: 4, flexShrink: 0,
      borderRight: `1px solid ${t.border}`
    }}>
      <div style={{ padding: "8px 12px 20px", display: "flex", justifyContent: "space-between" }}>
        <span style={{ fontFamily: "Georgia, serif", fontSize: 22, color: t.text, }}>
          <button onClick={() => setDark(d => !d)} title={dark ? "Light mode" : "Dark mode"}
            background: "none", border: "none", cursor: "pointer", fontSize: 17, pa

```

```

        color: t.muted, lineHeight: 1
    }}>{dark ? "Light" : "Dark"}</button>
</div>
{nav.map(item => (
    <button key={item.id} onClick={() => setActive(item.id)} style={{
        background: active === item.id ? t.navActive : "transparent",
        border: "none", borderRadius: 10, padding: "10px 14px",
        color: active === item.id ? t.navActiveTxt : t.navTxt,
        display: "flex", alignItems: "center", gap: 10,
        cursor: "pointer", fontSize: 14, textAlign: "left", transition: "all 0.
    }}>
        <span style={{ fontSize: 13 }}>{item.icon}</span>{item.label}
    </button>
    )}}
</div>
);
}

// Dashboard

function Dashboard({ habits, setHabits, reminder, setReminder, t }) {
    const days = last7Days();
    const doneToday = habits.filter(h => h.completions[todayKey]).length;
    const pct = habits.length ? Math.round((doneToday / habits.length) * 100) : 0;
    const [notifStatus, setNotifStatus] = useState(Notification.permission ?? "defa

    function toggle(id) {
        setHabits(prev => prev.map(h => {
            if (h.id !== id) return h;
            const c = { ...h.completions };
            if (c[todayKey]) delete c[todayKey]; else c[todayKey] = true;
            return { ...h, completions: c };
        }));
    }

    async function handleReminderToggle() {
        if (!reminder.enabled) {
            const perm = await requestNotificationPermission();
            setNotifStatus(perm);
            if (perm === "granted") {
                const updated = { ...reminder, enabled: true };
                setReminder(updated);
                scheduleReminder(updated.time, habits);
            } else if (perm === "denied") {
                alert("Notifications are blocked. Please enable them in your browser/devi
            } else if (perm === "unsupported") {
                alert("Your browser doesn't support notifications.");
            }
        }
    }
}

```

```

    }
  } else {
    cancelReminder();
    setReminder(r => ({ ...r, enabled: false }));
  }
}

function handleTimeChange(e) {
  const newTime = e.target.value;
  setReminder(r => ({ ...r, time: newTime }));
  if (reminder.enabled) scheduleReminder(newTime, habits);
}

const nextReminderText = () => {
  if (!reminder.enabled) return null;
  const [h, m] = reminder.time.split(":").map(Number);
  const next = new Date(); next.setHours(h, m, 0, 0);
  if (next <= new Date()) next.setDate(next.getDate() + 1);
  return next.toLocaleString("en-US", { weekday: "short", hour: "numeric", minu
};

return (
  <div style={{ padding: "32px 36px", flex: 1, overflowY: "auto", background: t
    <div style={{ display: "flex", justifyContent: "space-between", alignItems:
      <div>
        <h1 style={{ fontFamily: "Georgia, serif", fontSize: 42, color: t.text,
        <p style={{ color: t.muted, margin: "4px 0 0", fontSize: 14 }}>
          {today.toLocaleDateString("en-US", { weekday: "long", year: "numeric"
        </p>
      </div>
    </div>
    <div style={{ background: t.card, borderRadius: 14, padding: "16px 20px",
      <CircularProgress pct={pct} color={t.accent} t={t} />
      <div>
        <div style={{ color: t.text, fontWeight: 600, fontSize: 15 }}>Daily P
        <div style={{ color: t.muted, fontSize: 13, marginTop: 2 }}>{doneToda
      </div>
    </div>
  </div>

  <div style={{ display: "flex", flexDirection: "column", gap: 14 }}>
    {habits.map(h => {
      const done = !!h.completions[todayKey];
      const s = streak(h);
      return (
        <div key={h.id} style={{
          background: t.card, borderRadius: 14, borderLeft: `3px solid ${h.co
          padding: "18px 20px", display: "flex", alignItems: "center", gap: 1

```

```

        boxShadow: "0 1px 4px rgba(0,0,0,0.06)"
    }}>
    <button onClick={() => toggle(h.id)} style={{
        width: 40, height: 40, borderRadius: 10, border: "none",
        background: done ? h.color : t.cardAlt,
        color: "white", fontSize: 16, cursor: "pointer",
        display: "flex", alignItems: "center", justifyContent: "center",
        flexShrink: 0, transition: "all 0.2s"
    }}>{done ? "v" : ""}</button>
    <div style={{ flex: 1 }}>
        <div style={{ color: h.color, fontWeight: 600, fontSize: 16 }}>{h
        <div style={{ color: t.muted, fontSize: 12, marginTop: 2 }}>{s} d
    </div>
    <div style={{ display: "flex", gap: 8, alignItems: "center" }}>
        {days.map((d, i) => {
            const k = dateKey(d);
            const isToday = k === todayKey;
            const filled = !!h.completions[k];
            return (
                <div key={i} style={{
                    width: 30, height: 30, borderRadius: 8,
                    background: filled ? h.color : t.cardAlt,
                    display: "flex", alignItems: "center", justifyContent: "cen
                    fontSize: 11, color: filled ? "white" : t.faint,
                    border: isToday ? `1.5px solid ${h.color}` : "none",
                    fontWeight: isToday ? 700 : 400
                }}>
                    {d.getDate() < 10 ? `0${d.getDate()}` : d.getDate()}
                </div>
            );
        })}
    </div>
    </div>
    </div>

    { /* Credit */ }
    <div style={{ textAlign: "center", marginTop: 28, color: t.faint, fontSize:
        Made by Harkirat Singh Pawar
    </div>

    { /* Daily Reminder Card */ }
    <div style={{
        marginTop: 20, background: t.card, borderRadius: 14,
        padding: "16px 20px", boxShadow: "0 1px 4px rgba(0,0,0,0.06)"
    }}>

```

```

<div style={{ display: "flex", alignItems: "center", justifyContent: "spa
  <div style={{ display: "flex", alignItems: "center", gap: 12 }}>
    <div style={{ width: 36, height: 36, borderRadius: 10, background: t.
      <div>
        <div style={{ color: t.text, fontWeight: 500, fontSize: 14 }}>Daily
        <div style={{ color: t.muted, fontSize: 12 }}>
          {reminder.enabled ? `Next: ${nextReminderText()}` : "Get nudged t
        </div>
      </div>
    </div>
  </div>
  <Toggle on={reminder.enabled} onToggle={handleReminderToggle} t={t} />
</div>
{reminder.enabled && (
  <div style={{ marginTop: 14, display: "flex", alignItems: "center", gap
    <span style={{ color: t.subtle, fontSize: 13 }}>Remind me at</span>
    <input type="time" value={reminder.time} onChange={handleTimeChange}
      style={{
        background: t.cardAlt, border: `1px solid ${t.border2}`,
        borderRadius: 8, padding: "6px 10px", color: t.text,
        fontSize: 14, outline: "none", cursor: "pointer"
      }} />
    <span style={{ color: t.faint, fontSize: 12 }}>every day</span>
  </div>
)}
{notifStatus === "denied" && (
  <div style={{ marginTop: 10, color: "#ef4444", fontSize: 12 }}>
    (!) Notifications are blocked. Enable them in Settings -> Safari -> N
  </div>
)}
</div>
</div>
);
}

// Habits

function HabitsPage({ habits, setHabits, t }) {
  const [modal, setModal] = useState(false);
  const [name, setName] = useState("");
  const [color, setColor] = useState(COLORS[0]);
  const [customColor, setCustomColor] = useState("#ff69b4");

  function create() {
    if (!name.trim()) return;
    setHabits(prev => [...prev, { id: Date.now(), name: name.trim(), color, compl
    setName(""); setColor(COLORS[0]); setModal(false);
  }

```



```

return (
  <div style={{ padding: "32px 36px", flex: 1, overflowY: "auto", background: t
    <div style={{ display: "flex", justifyContent: "space-between", alignItems:
      <div>
        <h1 style={{ fontFamily: "Georgia, serif", fontSize: 42, color: t.text,
          <p style={{ color: t.muted, margin: "4px 0 0", fontSize: 14 }}>Design y
        </div>
        <button onClick={() => setModal(true)} style={{
          background: t.navActive, border: "none", borderRadius: 10,
          color: t.navActiveTxt, padding: "10px 18px", cursor: "pointer", fontSiz
        }}>+ New Habit</button>
      </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
      {habits.map(h => (
        <div key={h.id} style={{
          background: t.card, borderRadius: 14, borderLeft: `3px solid ${h.col
          padding: "16px 20px", display: "flex", alignItems: "center", gap: 14,
          boxShadow: "0 1px 4px rgba(0,0,0,0.06)"
        }}>
          <div style={{ width: 12, height: 12, borderRadius: "50%", background:
            <div style={{ flex: 1, color: t.text, fontWeight: 500, fontSize: 15 }
            <div style={{ color: t.muted, fontSize: 12 }}>{streak(h)} day streak<
            <button onClick={() => setHabits(prev => prev.filter(x => x.id !== h.
              background: "transparent", border: "none", color: t.faint,
              cursor: "pointer", fontSize: 16, padding: "2px 6px", borderRadius:
            }}>x</button>
          </div>
        </div>
      ))}
      {habits.length === 0 && (
        <div style={{ color: t.faint, fontSize: 15, padding: "40px 0", textAlig
          No habits yet. Create one to get started.
        </div>
      )}
    </div>
  )

  {modal && (
    <div style={{ position: "fixed", inset: 0, background: "rgba(0,0,0,0.5)",
      <div style={{ background: t.modalBg, borderRadius: 18, padding: 32, wid
        <div style={{ display: "flex", justifyContent: "space-between", align
          <h2 style={{ color: t.text, margin: 0, fontSize: 20, fontWeight: 60
            <button onClick={() => setModal(false)} style={{ background: "trans
          </div>
          <label style={{ color: t.subtle, fontSize: 13, fontWeight: 500 }}>Nam
          <input value={name} onChange={e => setName(e.target.value)}
            placeholder="e.g. Read 10 pages"
            onKeyDown={e => e.key === "Enter" && create()}

```

```

        style={{
          width: "100%", marginTop: 8, marginBottom: 20,
          background: t.inputBg, border: `1px solid ${t.border2}`,
          borderRadius: 10, padding: "11px 14px", color: t.text,
          fontSize: 14, outline: "none", boxSizing: "border-box"
        }} />
<label style={{ color: t.subtle, fontSize: 13, fontWeight: 500 }}>Col
<div style={{ display: "flex", flexWrap: "wrap", gap: 8, marginTop: 1
  {COLORS.map(c => (
    <button key={c} onClick={() => setColor(c)} style={{
      width: 34, height: 34, borderRadius: "50%", background: c, border
      cursor: "pointer", outline: color === c ? "2.5px solid " + t.te
      outlineOffset: 2, transform: color === c ? "scale(1.1)" : "scal
    }} />
  )))
</div>
<div style={{ display: "flex", alignItems: "center", gap: 10, marginB
  <input type="color" value={customColor}
    onChange={e => { setCustomColor(e.target.value); setColor(e.targe
      style={{ width: 44, height: 34, border: "none", borderRadius: 8,
    <span style={{ color: t.muted, fontSize: 13 }}>Custom Color</span>
  </div>
  <div style={{ display: "flex", gap: 10, justifyContent: "flex-end" }}
    <button onClick={() => setModal(false)} style={{ background: t.navA
    <button onClick={create} style={{ background: t.accentBtn, border:
  </div>
</div>
</div>
  )}
</div>
);
}

```

```
// Statistics
```

```

function Statistics({ habits, t }) {
  const last90 = useMemo(() => Array.from({ length: 90 }, (_, i) => getDaysAgo(89
  return (
    <div style={{ padding: "32px 36px", flex: 1, overflowY: "auto", background: t
      {habits.length === 0 && (
        <div style={{ color: t.faint, fontSize: 15, padding: "40px 0", textAlign:
      )}
      {habits.map(h => {
        const s = streak(h), ls = longestStreak(h);
        const r7 = rateForDays(h, 7), r30 = rateForDays(h, 30);
        const total = Object.values(h.completions).filter(Boolean).length;
        return (

```

```

<div key={h.id} style={{ marginBottom: 48 }}>
  <div style={{ display: "flex", alignItems: "center", gap: 10, marginB
    <div style={{ width: 12, height: 12, borderRadius: "50%", backgroun
      <h2 style={{ fontFamily: "Georgia, serif", fontSize: 30, color: t.t
    </div>
  <div style={{ display: "grid", gridTemplateColumns: "repeat(4,1fr)",
    [[["Current Streak", `${s} days`], ["Longest Streak", `${ls} days`]
      <div key={label} style={{ background: t.card, borderRadius: 12, p
        <div style={{ color: t.muted, fontSize: 12, marginBottom: 6 }}>
        <div style={{ color: t.text, fontSize: 20, fontWeight: 700 }}>{
          </div>
        </div>
      </div>
    </div>
  <div style={{ background: t.card, borderRadius: 12, padding: "20px 22
    <div style={{ color: t.subtle, fontSize: 13, marginBottom: 14 }}>La
    <div style={{ display: "flex", flexWrap: "wrap", gap: 5 }}>
      {last90.map((d, i) => (
        <div key={i} style={{
          width: 12, height: 12, borderRadius: "50%",
          background: h.completions[dateKey(d)] ? h.color : t.cardAlt
        </div>
      </div>
    </div>
    <div style={{ color: t.faint, fontSize: 12, marginTop: 12 }}>{total
  </div>
</div>
  );
  }}}
</div>
);
}

```

```
// Calendar
```

```

function Calendar({ habits, t }) {
  const [year, setYear] = useState(today.getFullYear());
  const [month, setMonth] = useState(today.getMonth());
  const [selected, setSelected] = useState(today);
  const monthNames = ["January", "February", "March", "April", "May", "June", "July", "A
  const firstDay = new Date(year, month, 1).getDay();
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const prevMonthDays = new Date(year, month, 0).getDate();
  const cells = [];
  for (let i = 0; i < firstDay; i++) cells.push({ day: prevMonthDays - firstDay +
  for (let i = 1; i <= daysInMonth; i++) cells.push({ day: i, current: true });
  for (let i = 1; i <= 42 - cells.length; i++) cells.push({ day: i, current: fals

```

```

function navMonth(dir) {
  let m = month + dir, y = year;
  if (m < 0) { m = 11; y--; } else if (m > 11) { m = 0; y++; }
  setMonth(m); setYear(y);
}
function getCompletions(d) {
  const k = `${year}-${String(month+1).padStart(2,'0')}-${String(d).padStart(2,
  return habits.filter(h => h.completions[k]);
}
const selKey = selected ? dateKey(selected) : null;
const selDone = selKey ? habits.filter(h => h.completions[selKey]) : [];
const isSelected = (d) => selected && selected.getFullYear() === year && select
const isTodayCell = (d) => year === today.getFullYear() && month === today.getM

return (
  <div style={{ padding: "32px 36px", flex: 1, overflowY: "auto", background: t
    <div style={{ marginBottom: 24 }}>
      <h1 style={{ fontFamily: "Georgia, serif", fontSize: 42, color: t.text, m
      <p style={{ color: t.muted, margin: "4px 0 0", fontSize: 14 }}>Your habit
    </div>
    <div style={{ display: "flex", alignItems: "center", justifyContent: "flex-
      <button onClick={() => navMonth(-1)} style={{ background: "none", border:
      <span style={{ color: t.text, fontWeight: 600, fontSize: 16 }}>{monthName
      <button onClick={() => navMonth(1)} style={{ background: "none", border:
    </div>
    <div style={{ display: "flex", gap: 16, marginBottom: 18, flexWrap: "wrap"
      {habits.map(h => (
        <div key={h.id} style={{ display: "flex", alignItems: "center", gap: 6,
          <div style={{ width: 9, height: 9, borderRadius: "50%", background: h
        </div>
      )})
    </div>
    <div style={{ background: t.card, borderRadius: 14, padding: "20px", boxSha
      <div style={{ display: "grid", gridTemplateColumns: "repeat(7,1fr)", text
        [{"SUN","MON","TUE","WED","THU","FRI","SAT"].map(d => (
          <div key={d} style={{ color: t.muted, fontSize: 11, fontWeight: 600,
        )})
      </div>
      <div style={{ display: "grid", gridTemplateColumns: "repeat(7,1fr)", gap:
        {cells.map((cell, i) => {
          if (!cell.current) return <div key={i} style={{ padding: "10px 4px",
          const completions = getCompletions(cell.day);
          const pct = habits.length ? Math.round((completions.length / habits.l
          const sel = isSelected(cell.day);
          const tod = isTodayCell(cell.day);
          return (
            <div key={i} onClick={() => setSelected(new Date(year, month, cell.

```

```

padding: "8px 4px 6px", textAlign: "center", cursor: "pointer",
borderRadius: 10, border: sel ? `1.5px solid ${t.accent}` : "1.5px solid transparent",
transition: "background 0.15s", minHeight: 54
}}>
<div style={{
  width: 26, height: 26, borderRadius: "50%", margin: "0 auto 4px auto",
  background: tod ? t.accent : "transparent",
  display: "flex", alignItems: "center", justifyContent: "center",
  color: tod ? "white" : t.text, fontSize: 14, fontWeight: tod ? "bold" : "normal",
}}>{cell.day}</div>
{completions.length > 0 && (
  <>
    <div style={{ display: "flex", justifyContent: "center", gap: 10 }}>
      {completions.slice(0, 3).map(h => <div key={h.id} style={{
        width: 20, height: 20, borderRadius: "50%", background: h.done ? t.accent : t.card,
        display: "flex", alignItems: "center", justifyContent: "center",
        color: h.done ? "white" : t.text, fontSize: 10,
      }}>{h.day}</div>)}
      <div style={{ color: t.muted, fontSize: 10 }}>{pct}%</div>
    </div>
  </div>
);
)}}
</div>
</div>
{selected && (
  <div style={{ background: t.card, borderRadius: 14, padding: "18px 22px",
    display: "flex", gap: 10, alignIt: "center" }}>
    <div style={{ color: t.text, fontWeight: 600, fontSize: 16 }}>
      {selected.toLocaleDateString("en-US", { weekday: "long", year: "numeric", month: "long", day: "numeric" })}
    </div>
    <button onClick={() => setSelected(null)} style={{ background: "none", border: "none", padding: 0 }}>
      </div>
    <div style={{ color: t.muted, fontSize: 13 }}>{selDone.length} of {habits.length}</div>
    {selDone.length > 0 && (
      <div style={{ display: "flex", flexWrap: "wrap", gap: 8, marginTop: 10 }}>
        {selDone.map(h => (
          <div key={h.id} style={{ background: t.cardAlt, borderRadius: 8, padding: 5 }}>
            <div style={{ width: 6, height: 6, borderRadius: "50%", background: h.done ? t.accent : t.card, display: "flex", alignIt: "center", justifyContent: "center" }}>
              </div>
            {h.day}
          </div>
        ))}
      </div>
    )}
  </div>
)}
</div>
);
}

```

```
// App

export default function App() {
  const [active, setActive] = useState("dashboard");
  const [habits, setHabits] = useState(() => loadHabits() ?? DEFAULT_HABITS);
  const [dark, setDarkRaw] = useState(() => loadDarkMode());
  const [reminder, setReminderRaw] = useState(() => loadReminder());

  const t = useMemo(() => makeTheme(dark), [dark]);

  function setDark(val) {
    const next = typeof val === "function" ? val(dark) : val;
    setDarkRaw(next);
    try { localStorage.setItem("ritual-darkmode", JSON.stringify(next)); } catch {}
  }

  function setReminder(val) {
    setReminderRaw(prev => {
      const next = typeof val === "function" ? val(prev) : val;
      try { localStorage.setItem("ritual-reminder", JSON.stringify(next)); } catch {}
      return next;
    });
  }

  useEffect(() => { saveHabits(habits); }, [habits]);

  // Re-schedule reminder if it was enabled on last session
  useEffect(() => {
    if (reminder.enabled && Notification.permission === "granted") {
      scheduleReminder(reminder.time, habits);
    }
  }, []);

  return (
    <div style={{ display: "flex", height: "100vh", background: t.bg, fontFamily:
      <Sidebar active={active} setActive={setActive} dark={dark} setDark={setDark}
      {active === "dashboard" && <Dashboard habits={habits} setHabits={setHabits}
      {active === "habits" && <HabitsPage habits={habits} setHabits={setHabits}
      {active === "statistics" && <Statistics habits={habits} t={t} />
      {active === "calendar" && <Calendar habits={habits} t={t} />
    </div>
  );
}
```